🚂 Railway Deployment Guide

Complete step-by-step guide for deploying AuthiChain to Railway.

🎯 Why Railway?

Pros:
- ✅ **Simplest deployment** - Click-button deploys
- ✅ **Built-in PostgreSQL** - Database included
- ✅ **Fair pricing** - Pay for what you use
- ✅ **Zero cold starts** - Always-on containers
- ✅ **Persistent storage** - Unlike serverless
- ✅ **One-click database backups** - Built-in
- ✅ **Great developer experience** - Intuitive UI

Cons:
- ❌ More expensive than Vercel at scale
- ❌ Manual scaling (not auto-scaling)

Pricing:
- **Starter:** $5/month (500 hours execution time)
- **Developer:** $20/month (2000 hours)
- **Team:** Custom pricing

Free Trial: $5 credit (no credit card required)

📋 Prerequisites

Before starting:
- ✅ GitHub account with AuthiChain repo
- ✅ All API keys configured (see API_KEYS_SETUP.md (./API_KEYS_SETUP.md))
- ✅ All 15 Stripe Price IDs created (see STRIPE_PRICE_SETUP.md (./STRIPE_PRICE_SETUP.md))
- ✅ Domain name (optional but recommended)

🚀 Deployment Steps

Step 1: Push Code to GitHub

1. **Initialize Git (if not already done):**

```
bash
  cd /path/to/authichain_premium/app
  git init
  git add .
  git commit -m "Initial commit - AuthiChain production ready"
```

2. **Create GitHub Repository:**
   - Go to github.com/new (https://github.com/new)
   - Repository name: `authichain`
   - Visibility: Private (recommended)
   - Click “Create repository”

3. **Push to GitHub:**
   ```bash
   git remote add origin https://github.com/YOUR_USERNAME/authichain.git
   git branch -M main
   git push -u origin main
   ```

Step 2: Create Railway Account

1. **Sign Up:**
   - Visit railway.app (https://railway.app)
   - Click “Start a New Project”
   - Choose “Login with GitHub”
   - Authorize Railway

2. **Verify Account:**
   - Add phone number (for $5 free credit)
   - Or add payment method for full access

Step 3: Create New Project

1. **Create Project:**
   - Click “New Project”
   - Select “Deploy from GitHub repo”

2. **Select Repository:**
   - Choose `authichain` repository
   - Railway will analyze the repo

3. **Configure Root Directory:**
   - If your Next.js app is in `app` folder:
   - Click “Settings” → “Root Directory”
   - Set to: `app`
   - Click “Save”

Step 4: Add PostgreSQL Database

Railway makes this super easy!

1. **In Your Project:**
   - Click “New” button
   - Select “Database”
   - Choose “PostgreSQL”

2. **Database Created!**
   - Railway provisions PostgreSQL instance
   - Takes ~30 seconds
   - No configuration needed!

3. **Get Connection String:**
   - Click on the PostgreSQL service
   - Go to “Connect” tab
   - Copy the “Postgres Connection URL”
   `postgresql://postgres:password@containers-us-west-xxx.railway.app:5432/railway`

4. **Automatic DATABASE_URL:**
   - Railway automatically adds `DATABASE_URL` to your app
   - No manual configuration needed!
   - Verify in “Variables” tab of your service

Step 5: Configure Environment Variables

1. **Click on Your App Service:**
   - (Not the database)
   - Go to “Variables” tab

2. **Add Variables:**
   - Click “+ New Variable”
   - Or use “RAW Editor” for bulk add

Method A: Add One-by-One

Click “+ New Variable” for each:

```
NEXTAUTH_SECRET=Xy9Z8vW+eT6uRs5qPn4mKj3hGf2dCa1b
STRIPE_SECRET_KEY=sk_live_51...
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY=pk_live_51...
STRIPE_WEBHOOK_SECRET=whsec_...
NFT_STORAGE_API_KEY=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
NEXT_PUBLIC_SITE_URL=https://authichain.up.railway.app
NODE_ENV=production
```

Method B: RAW Editor (Faster!)

1. Click “RAW Editor” button
2. Paste all variables at once:

```
NEXTAUTH_SECRET=Xy9Z8vW+eT6uRs5qPn4mKj3hGf2dCa1b
NEXTAUTH_URL=${{RAILWAY_PUBLIC_DOMAIN}}
STRIPE_SECRET_KEY=sk_live_51...
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY=pk_live_51...
STRIPE_WEBHOOK_SECRET=whsec_...
NFT_STORAGE_API_KEY=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
NEXT_PUBLIC_SITE_URL=https://${{RAILWAY_PUBLIC_DOMAIN}}
NODE_ENV=production

# Stripe Price IDs (All 15)
STRIPE_STARTER_MONTHLY_PRICE_ID=price_...
STRIPE_STARTER_ANNUAL_PRICE_ID=price_...
STRIPE_STARTER_LIFETIME_PRICE_ID=price_...

STRIPE_PROFESSIONAL_MONTHLY_PRICE_ID=price_...
STRIPE_PROFESSIONAL_ANNUAL_PRICE_ID=price_...
STRIPE_PROFESSIONAL_LIFETIME_PRICE_ID=price_...

STRIPE_BUSINESS_MONTHLY_PRICE_ID=price_...
STRIPE_BUSINESS_ANNUAL_PRICE_ID=price_...
STRIPE_BUSINESS_LIFETIME_PRICE_ID=price_...

STRIPE_ENTERPRISE_MONTHLY_PRICE_ID=price_...
STRIPE_ENTERPRISE_ANNUAL_PRICE_ID=price_...
STRIPE_ENTERPRISE_LIFETIME_PRICE_ID=price_...

STRIPE_WHITE_LABEL_MONTHLY_PRICE_ID=price_...
STRIPE_WHITE_LABEL_ANNUAL_PRICE_ID=price_...
STRIPE_WHITE_LABEL_LIFETIME_PRICE_ID=price_...
```

1. Click “Update Variables”

> 💡 **Pro Tip:** `${{RAILWAY_PUBLIC_DOMAIN}}` is automatically replaced with your Railway URL!

Step 6: Configure Build Settings

1. **Build Command:**
   - Railway auto-detects Next.js
   - Default: `npm run build`
   - If custom build needed, go to Settings → Build

2. **Start Command:**
   - Default: `npm start`
   - For Next.js production: `npm run start`

3. **Node Version:**
   - Add to `package.json`:
   `json`
   `{`
   `"engines": {`
   `"node": "18.x"`
   `}`
   `}`

Step 7: Run Database Migrations

Railway doesn't auto-run Prisma migrations, so we'll use a build script:

Option A: Add Build Script (Recommended)

1. **Update `package.json`:**
   ```
   json
   {
     "scripts": {
       "build": "prisma generate && prisma db push && next build",
       "start": "next start"
     }
   }
   ```

2. **Commit and push:**
   ```
   bash
   git add package.json
   git commit -m "Add Prisma migration to build"
   git push
   ```

3. **Railway will auto-deploy** and run migrations!

Option B: Manual Migration (One-Time)

1. **Install Railway CLI:**
   ```
   bash
   npm i -g @railway/cli
   ```

2. **Login:**
   ```
   bash
   railway login
   ```

3. **Link Project:**
   ```
   bash
   railway link
   ```
   - Select your project
   - Select your app service (not database)

4. **Run Migration:**
   ```
   bash
   railway run npx prisma db push
   ```

Step 8: Deploy Application

Railway automatically deploys on every push!

1. **First Deployment:**
   - Railway detected your repo
   - Automatically starts building
   - Watch logs in real-time

2. **Build Process:**
   - Install dependencies

- Run Prisma generation
- Push database schema
- Build Next.js app
- Start server

3. **Get Your URL:**
   - Once deployed, go to “Settings” tab
   - Click “Generate Domain”
   - You’ll get: `https://authichain-production.up.railway.app`
   - Visit and test!

Step 9: Configure Custom Domain

1. **In Railway Dashboard:**
   - Click on your app service
   - Go to “Settings” tab
   - Scroll to “Domains”
   - Click “Custom Domain”

2. **Add Your Domain:**
   - Enter: `authichain.com`
   - Railway shows you DNS records to add

3. **DNS Configuration:**

At your domain registrar, add:

A Record:
```
Type: A
  Name: @
  Value: [IP provided by Railway]
  TTL: 3600
```

CNAME for www:
```
Type: CNAME
  Name: www
  Value: [domain provided by Railway]
  TTL: 3600
```

1. **Wait for Verification:**
   - Usually takes 5-60 minutes
   - Railway shows “Verified” when ready
   - SSL certificate auto-provisioned!

2. **Update Environment Variables:**
   - Go to “Variables” tab
   - Update `NEXTAUTH_URL` to your custom domain
   - Update `NEXT_PUBLIC_SITE_URL` to your custom domain
   - Railway will auto-redeploy

Step 10: Configure Stripe Webhook

1. **Get Your Live URL:**
   `https://authichain.com`

2. **Update Stripe Webhook:**
   - Go to Stripe Dashboard (https://dashboard.stripe.com)
   - Navigate to Developers → Webhooks
   - Find your webhook endpoint
   - Update endpoint URL to:
   `https://authichain.com/api/stripe/webhook`

3. **Test Webhook:**
   - In Stripe Dashboard, click “Send test webhook”
   - Select `checkout.session.completed`
   - Should return 200 OK

⚙️ Advanced Configuration

Enable Database Backups

1. **In Railway Dashboard:**
   - Click on PostgreSQL service
   - Go to “Data” tab
   - Click “Backups”
   - Enable automatic backups
   - Choose frequency: Daily (recommended)

2. **Manual Backup:**
   ```bash
   railway run pg_dump $DATABASE_URL > backup.sql
   ```

Configure Health Checks

1. **Create Health Check Endpoint:**
   ```typescript
   // app/api/health/route.ts
   export async function GET() {
     return Response.json({
       status: 'healthy',
       timestamp: new Date().toISOString()
     });
   }
   ```

2. **Enable in Railway:**
   - Go to Settings → Health Check
   - Path: `/api/health`
   - Interval: 60 seconds
   - Timeout: 10 seconds

Set Up Monitoring

Railway provides built-in monitoring:

1. **View Metrics:**
   - Click on your service
   - Go to “Metrics” tab
   - See:
     - CPU usage
     - Memory usage
     - Network traffic
     - Response times
2. **Set Up Alerts:**
   - Railway Pro plan includes alerts
   - Configure via Webhooks
   - Get notified of:
     - High CPU usage
     - Memory leaks
     - Deployment failures

Configure Log Retention

1. **View Logs:**
   - Click on service
   - Go to “Logs” tab
   - See real-time logs
2. **Filter Logs:**
   - Use search bar
   - Filter by severity
   - Export logs (Pro plan)
3. **Persistent Logging:**

```
typescript
  // Use structured logging
  console.log(JSON.stringify({
    level: 'info',
    message: 'User registered',
    userId: user.id,
    timestamp: new Date().toISOString()
  }));
```

Scale Your Application

1. **Vertical Scaling (Increase Resources):**
   - Click on service

- Go to “Settings”
- Scroll to “Resources”
- Increase:
  - RAM: 512MB → 8GB
  - CPU: 0.5 vCPU → 8 vCPU

2. **Horizontal Scaling (Multiple Instances):**
   - Railway Pro plan
   - Settings → Replicas
   - Set number of instances

🔄 Continuous Deployment

Railway automatically deploys on git push:

1. **Make Changes:**
   ```bash
   # Edit code
   git add .
   git commit -m "Add new feature"
   git push
   ```

2. **Automatic Deployment:**
   - Railway detects push
   - Starts new build
   - Zero-downtime deployment
   - Previous version kept running until new one is healthy

3. **Deployment Branches:**
   - Settings → Service → GitHub Branch
   - Change from `main` to `production` if needed

🧪 Testing Production Deployment

1. Health Check

```
curl https://authichain.com/api/health
```

Expected response:

```
{"status":"healthy","timestamp":"2025-10-19T12:00:00.000Z"}
```

2. Database Connection

```
railway connect PostgreSQL
```

Then in psql:

```
\dt -- List tables
SELECT COUNT(*) FROM users;
```

3. Test User Flow

1. Visit your site
2. Register new user
3. Log in
4. Subscribe to a tier
5. Mint an NFT
6. Verify everything works!

🚨 Troubleshooting

Build Fails

Error: `Build failed with exit code 1`

Solution:
1. Check build logs in Railway dashboard
2. Verify all dependencies in `package.json`
3. Test build locally:
`bash`
`npm run build`
4. Check Node version compatibility

Database Connection Issues

Error: `P1001: Can't reach database server`

Solution:
1. Verify PostgreSQL service is running
2. Check `DATABASE_URL` in variables
3. Ensure services are in same project
4. Test connection:
`bash`
`railway run npx prisma db pull`

Deployment Timeout

Error: `Deployment exceeded time limit`

Solution:
1. Increase timeout in Settings → Deploy
2. Optimize build:
`json`

```
  {
    "scripts": {
      "build": "prisma generate && next build --experimental-build-mode=generate"
    }
  }
```

Out of Memory

Error: `JavaScript heap out of memory`

Solution:
1. Increase Node memory:
```
json
  {
    "scripts": {
      "build": "NODE_OPTIONS='--max-old-space-size=4096' next build"
    }
  }
```
2. Or increase Railway service RAM in Settings → Resources

Webhook Not Working

Error: Stripe webhook returns 404

Solution:
1. Verify route exists: `app/api/stripe/webhook/route.ts`
2. Check Railway logs for errors
3. Test locally with Stripe CLI:
```
bash
  stripe listen --forward-to https://authichain.com/api/stripe/webhook
```

🔒 Security Best Practices

1. Use Private Variables

Railway variables are encrypted at rest:

1. Mark sensitive vars as “Shared”
2. Never log environment variables
3. Use `.gitignore` for local `.env` files

2. Enable 2FA

1. Go to Account Settings
2. Enable two-factor authentication
3. Use authenticator app

3. IP Whitelisting (Database)

For extra security:

1. Get Railway NAT IPs
2. Whitelist only those IPs in your firewall
3. Restrict public database access

4. Regular Backups

1. Enable automatic backups
2. Test restore process monthly
3. Store backups off-Railway

📊 Cost Optimization

Monitor Usage

1. **Check Usage:**
   - Railway Dashboard → Usage
   - See:
     - Execution time
     - Memory usage
     - Network egress
2. **Optimize Costs:**
   - Reduce memory if possible
   - Use caching to reduce CPU
   - Optimize database queries

Estimated Costs

Usage Level	Estimated Cost
Hobby/Testing	$5-10/month
Small Production	$20-50/month
Medium Traffic	$50-150/month
High Traffic	$150-500/month

🔧 Railway CLI Commands

Useful Commands:

```
# Install CLI
npm i -g @railway/cli

# Login
railway login

# Link project
railway link

# View logs
railway logs

# Run command in Railway environment
railway run [command]

# Connect to database
railway connect PostgreSQL

# List projects
railway list

# Deploy specific branch
railway up

# Open project in browser
railway open
```

📚 Additional Resources

- Railway Documentation (https://docs.railway.app)
- Railway CLI Reference (https://docs.railway.app/develop/cli)
- Railway Templates (https://railway.app/templates)
- Railway Discord Community (https://discord.gg/railway)

✅ Deployment Checklist

- [ ] GitHub repo created and pushed
- [ ] Railway project created
- [ ] PostgreSQL database added
- [ ] All environment variables configured
- [ ] Database migrations run successfully
- [ ] Application deployed and accessible
- [ ] Custom domain configured (if applicable)
- [ ] SSL certificate active
- [ ] Stripe webhook updated and tested

- [ ] Health check endpoint working
- [ ] Database backups enabled
- [ ] Test user registration
- [ ] Test subscription checkout
- [ ] Test NFT minting
- [ ] Monitoring and alerts configured
- [ ] Complete PRODUCTION_CHECKLIST.md (./PRODUCTION_CHECKLIST.md)

Congratulations! AuthiChain is now live on Railway! 🚂🎉